WEEK 7 — WEB APPLICATION VULNERABILITY TESTING

Vulnerability Assessment & Penetration Testing (VAPT) with Burp Suite Community Edition

Student	SAMYUKTHA S.
Internship ID	DG/SEPTEMBER/CYBER/102
Programme	DG Interns Hub – Cybersecurity Internship
Target	http://127.0.0.1/dvwa/ and local HTTPS endpoint
Environment	Kali Linux • Local DVWA lab • Burp Suite Community Edition
Report date	18 October 2026

Assessment boundary

All practical testing documented here was performed against an intentionally vulnerable application in a controlled local lab. The report records observed behavior and evidence; it does not represent a production-system audit. Site-map entries, setup screenshots, and request changes are not treated as vulnerabilities unless a test result supports that conclusion.

Practical focus: interception • attack-surface mapping • SQL injection • reflected and stored XSS • request comparison • endpoint inventory

Report Overview

This report documents the Week 7 web-application testing exercise using Burp Suite Community Edition. The practical work concentrated on observing browser traffic, identifying the routes exposed by local DVWA, demonstrating three input-handling issues in the training configuration, and preserving request/response evidence. The report also records tool configurations that were not carried through to a confirmed security finding.

Contents

Section	Coverage
1–2	Objective, scope, authorization and limits
3–4	Lab environment, tools and methodology
5–6	Proxy interception, HTTPS observation and reconnaissance
7	SQL injection, reflected XSS, stored XSS, Intruder setup and parameter comparison
8	Endpoint discovery and route inventory
9–11	Findings register, remediation and validation plan
12–14	Evidence index, learning outcomes, conclusion and references

Evidence rule

A result is labelled “demonstrated in local lab” only when the captured behavior supports it. A route in the site map indicates that it was observed during browsing; it does not prove that the route is vulnerable. Intruder payload-position setup is configuration evidence only, because no guessing attack was run. Changing an ID and receiving a response is not sufficient to prove broken access control.

1. Objective, Scope and Authorization

The objective was to practice a repeatable web-application testing workflow: capture requests, understand parameters, map reachable application paths, submit controlled test inputs, review the response, and document the result with a remediation recommendation.

Assessment scope

Item	Definition
Target	Local Damn Vulnerable Web Application (DVWA) at 127.0.0.1
Access path	http://127.0.0.1/dvwa/; local HTTPS navigation was also observed
Security setting	DVWA Low for the demonstrated SQLi/XSS cases
Allowed activity	Manual browsing, Burp Proxy, Target site map, Repeater and Intruder position configuration
Out of scope	Public websites, third-party systems, production credentials and unapproved targets

2. Lab Environment and Tools

Component	Purpose in this exercise
Kali Linux	Local testing environment for browser and Burp Suite
DVWA	Intentionally vulnerable training application used for controlled demonstrations
Burp Proxy / HTTP history	Captured browser requests and supported inspection of request methods, parameters and responses
Burp Target → Site map	Organized routes observed while browsing the application
Burp Repeater	Sent a modified request and allowed response comparison
Burp Intruder	Documented a selected username payload position; no attack was launched
Screenshots and notes	Preserved evidence for report statements and route inventory

3. Methodology

The testing process followed a simple sequence so that each observation could be connected to the request that produced it. The work was restricted to the local training application throughout.

1. Prepare the lab

Start the local DVWA service and confirm that the application can be opened in the browser.

2. Configure interception

Use Burp Proxy with the local browser and verify that browser requests appear in HTTP history.

3. Observe HTTPS navigation

Review the local HTTPS traffic shown in the captured evidence; do not infer a certificate configuration detail that the screenshot does not establish.

4. Map the application

Browse the modules and review Target → Site map to record observed routes and their apparent roles.

5. Establish a baseline

Capture a normal SQLi-module request before submitting the controlled test input.

6. Run limited test cases

Use DVWA Low to demonstrate SQL injection and reflected/stored XSS behavior.

7. Compare requests

Send a changed ID parameter through Repeater and record that a response was returned.

8. Preserve evidence

Save screenshots and notes near the related task; separate confirmed results from setup-only observations.

Interpretation principle

The report avoids claiming that a security control was fixed or that a vulnerability was fully remediated, because no separate post-fix validation is documented for this Week 7 lab. Recommendations are presented as next steps, not as completed changes.

4. Proxy Setup and Interception

Burp Suite was used to capture a local DVWA login POST request and inspect traffic generated during navigation. The login request demonstrates that form submission traffic was available for review in Burp. When assessing a login request, the method, path, content type and parameter names are useful first checks; sensitive values should not be copied into a public report.

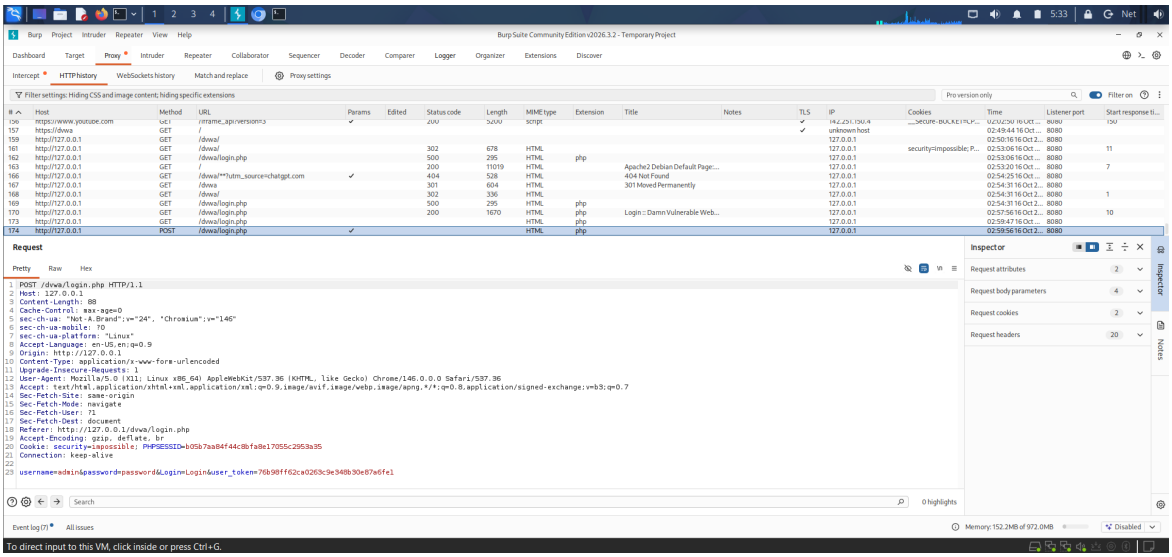


Figure 1. Local DVWA login POST request captured in Burp.

HTTPS observation

The supplied evidence records HTTPS traffic to the local DVWA instance. HTTPS interception allows a tester to inspect the HTTP request/response after the browser and proxy establish the appropriate trust relationship. This screenshot is used as evidence of observed traffic only; it is not presented as a standalone certificate-validation test.

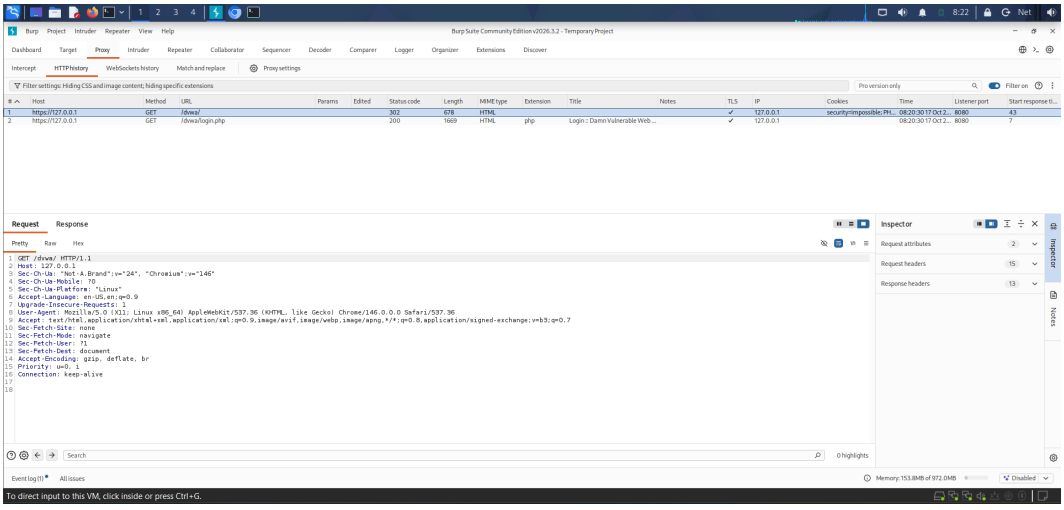


Figure 2. HTTPS traffic to the local DVWA instance observed in Burp.

5. Reconnaissance and Attack-Surface Mapping

Manual browsing and Burp Target → Site map were used to build an initial inventory of the local DVWA application. The purpose of this stage was to understand which functional areas were exposed and where later manual tests could be performed. It was not a claim of exhaustive crawling or directory brute forcing.

Observed application areas

Area	Observed path	Reason to record
Authentication	/dvwa/login.php	Login form and POST request handling
Configuration	/dvwa/security.php	Security-level configuration page
SQL injection	/dvwa/vulnerabilities/sqli/	Input field used for SQLi demonstration
Reflected XSS	/dvwa/vulnerabilities/xss_r/	Input reflected in a page response
Stored XSS	/dvwa/vulnerabilities/xss_s/	Guestbook entry stored and rendered later
File upload	/dvwa/vulnerabilities/upload/	Module observed during browsing
CSRF	/dvwa/vulnerabilities/csrf/	Module observed during browsing

Reconnaissance limitations

The inventory was assembled by manual browsing and the built-in site map. No dirsearch or Gobuster run is documented, so the report does not claim directory brute-force coverage. A path appearing in the map confirms observation, not exploitability; each security issue needs its own test evidence.

5.1 Site Map Evidence and Interpretation

The screenshot below supports the route inventory by showing the local host and DVWA paths within Burp Target → Site map. This view is useful for organizing requests by host and application path, then selecting a route for manual review.

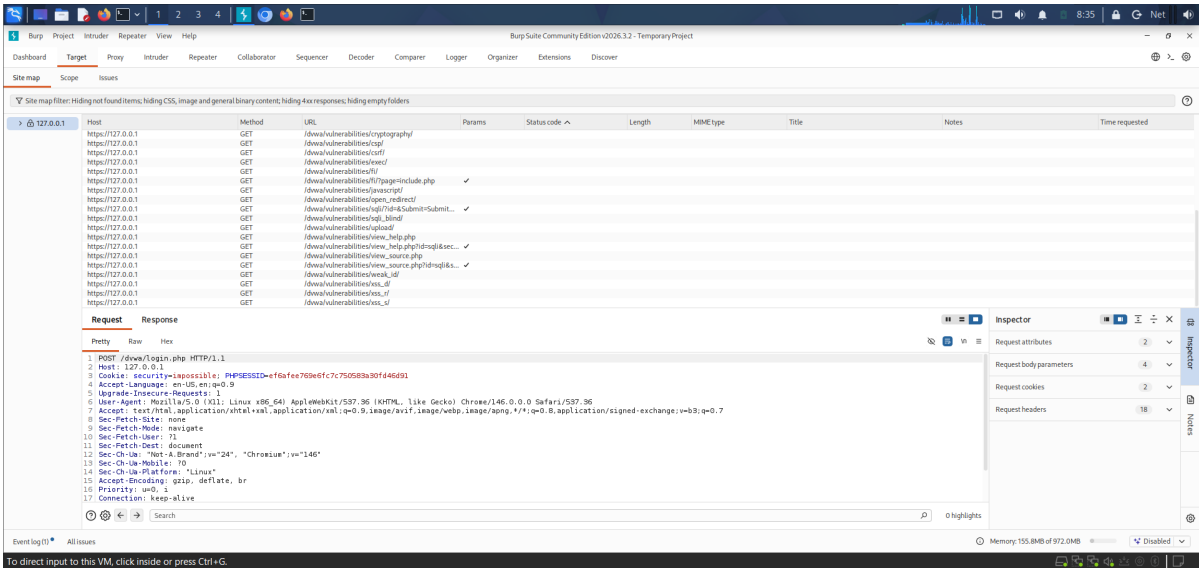


Figure 3. Burp Target → Site map for the local DVWA application.

How the map was used

- Group observed requests by the local host and path.
- Identify the login and security pages separately from the vulnerability-training modules.
- Select the SQLi and XSS modules for the documented controlled test cases.
- Record additional observed modules as inventory items only, unless separate validation evidence exists.

This distinction keeps reconnaissance useful without overstating results: discovery identifies where to investigate; validation determines whether the behavior represents a vulnerability.

6.1 SQL Injection — DVWA Low

Status: demonstrated in the local training lab.

A baseline request was captured for the SQL Injection module. A controlled Boolean-style input was then submitted in the DVWA Low configuration. The crafted input caused the page to display multiple user records, which is consistent with SQL injection behavior in this intentionally vulnerable application. The result is limited to the local lab and does not establish the impact of a similarly named issue in any other system.

Reproduction summary

- Open the local DVWA SQL Injection module and confirm the training security level is Low.
- Submit a normal ID value and preserve the baseline request/response.
- Submit the controlled test input used in the lab and observe the response behavior.
- Compare the two responses and preserve the screenshot showing the multiple-record result.

Potential impact in a real application

If untrusted input is concatenated into a database query, an attacker may be able to alter query logic. Depending on query structure and database privileges, consequences can include unauthorized data retrieval or modification. The exact impact cannot be inferred from the DVWA demonstration alone.

Recommended remediation

- Use parameterized queries or prepared statements rather than concatenating user input into SQL.
- Apply least privilege to database accounts and avoid exposing detailed database errors.
- Validate input according to the expected type and length; validation supplements but does not replace parameterization.
- Add regression tests for unexpected input and SQL metacharacters.

6.1 SQL Injection Evidence

The two evidence items below are kept together to show the baseline and the resulting behavior. Read them as a comparison: the baseline establishes ordinary application behavior, while the second screenshot documents the response after the controlled test input.

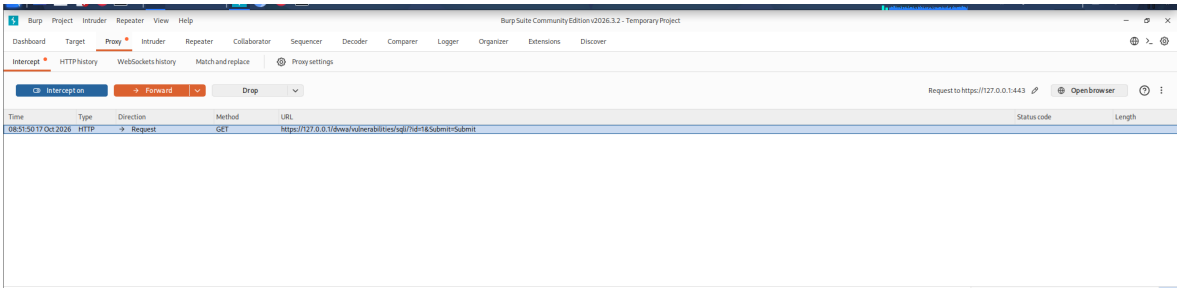


Figure 4. Baseline SQL Injection request.

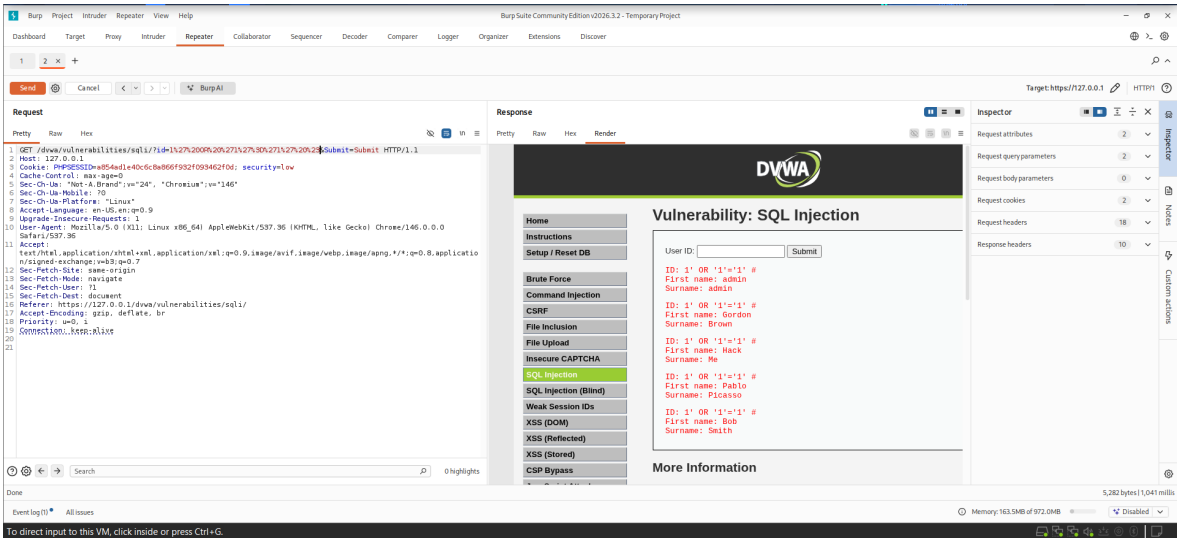


Figure 5. Crafted input returned multiple user records in local DVWA.

Evidence interpretation

The response difference supports the local SQL injection demonstration. A production assessment would additionally document the exact request parameter, repeatability, affected query behavior, application context and any safe impact validation approved by the system owner. Those extra production checks are not claimed here.

6.2 Reflected Cross-Site Scripting (XSS)

Status: demonstrated in the local training lab.

A JavaScript alert test payload was submitted through DVWA’s Reflected XSS module. The browser displayed an alert, and the associated request was captured in Burp. This supports script execution in the intentionally vulnerable local page. The finding is not evidence that an external application is affected.

Reproduction summary

- Open the Reflected XSS module at DVWA Low.
- Submit the lab-only alert payload through the module form.
- Observe whether the browser alert appears.
- Review the matching request and response in Burp and save evidence.

Potential impact

Reflected XSS may allow attacker-controlled script to execute in a victim’s browser under the application origin. Depending on the page and session model, this can support page manipulation, deceptive content or phishing. Impact should be evaluated in the actual application context and not assumed from the alert alone.

Recommended remediation

- Apply context-aware output encoding where untrusted values are rendered.
- Use safe templating and DOM APIs; avoid inserting untrusted strings as HTML or script.
- If rich text is required, use a maintained allowlist sanitizer.
- Use a restrictive Content Security Policy as defense in depth, not as the primary fix.

6.2 Reflected XSS Evidence

The browser screenshot records the alert result. The Burp screenshot records the associated HTTP evidence, helping connect the observed browser behavior to the submitted request.

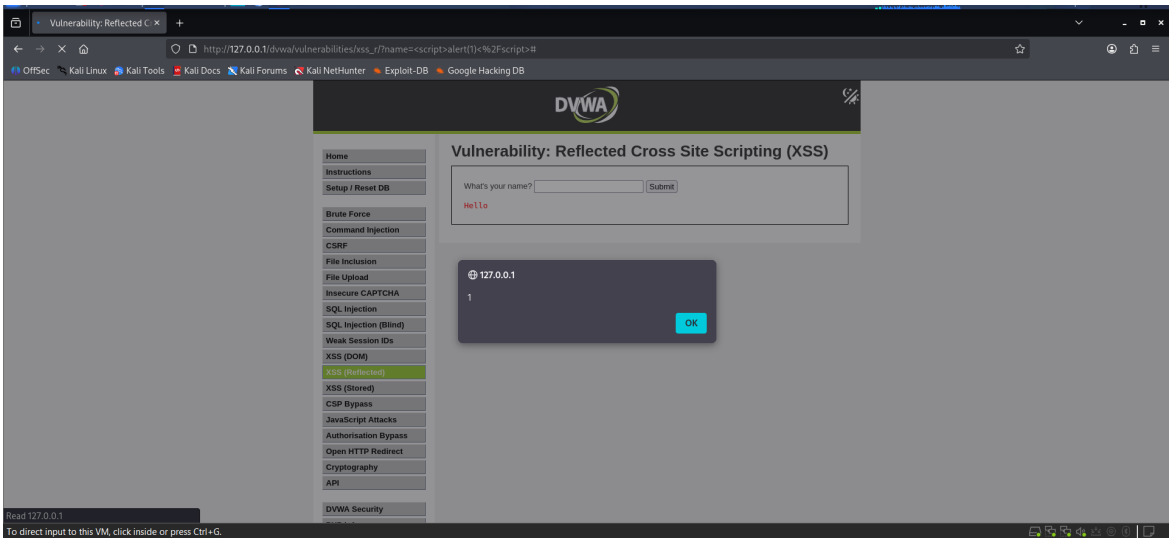


Figure 6. Reflected XSS browser confirmation in local DVWA.

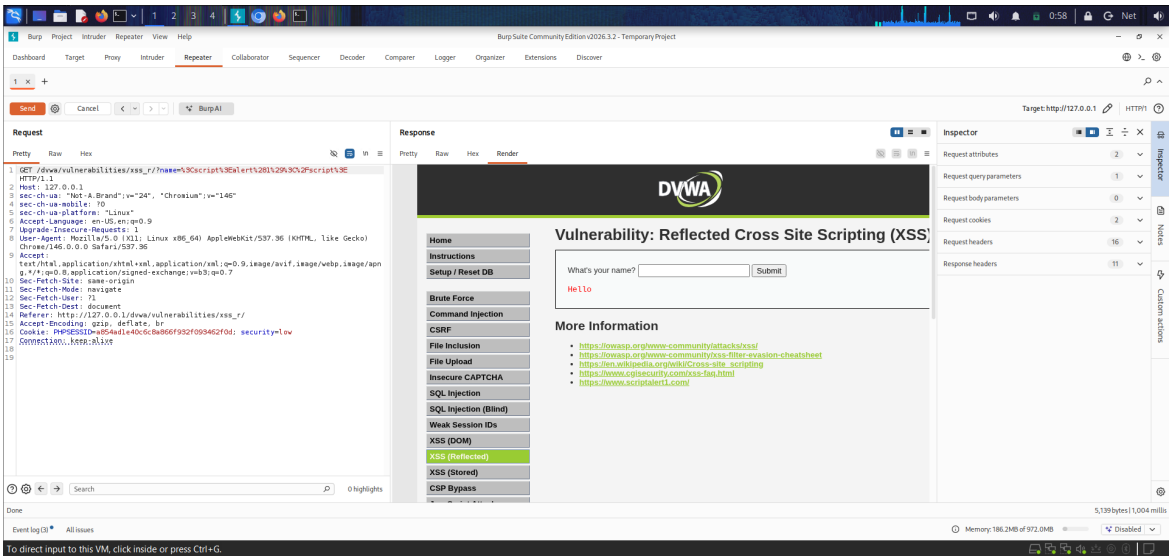


Figure 7. Burp request/response evidence for the reflected input.

The alert is a practical confirmation of script execution in this lab. A full application assessment would also identify the precise output context, test encoding behavior, and validate the remediation with the same test case after a fix.

6.3 Stored Cross-Site Scripting (XSS)

Status: demonstrated in the local training lab.

A test message was submitted through the DVWA guestbook with a JavaScript alert payload. The entry was stored and later triggered an alert when rendered. This differs from reflected XSS because the test content remained in application data and executed during a later page view. The demonstration was performed only in local DVWA.

Reproduction summary

- Open the Stored XSS / guestbook module at DVWA Low.
- Submit a clearly identifiable test name and the lab-only alert payload as the message.
- Return to or refresh the guestbook view and observe whether the stored entry triggers the alert.
- Review the POST request and response in Burp and preserve the evidence.

Potential impact

Stored XSS can affect people who later view the saved content. Depending on the application origin, affected page and session protections, it may enable persistent page manipulation, deceptive messages or access to information available to scripts in that origin.

Recommended remediation

- Encode untrusted stored content when outputting it, using the correct HTML, attribute or JavaScript context.
- Use safe rendering APIs and sanitize rich HTML with a maintained allowlist sanitizer only when rich text is necessary.
- Review existing stored entries and add regression tests for both saving and rendering user content.

6.3 Stored XSS Evidence

The first screenshot documents the browser-side behavior. The second captures the related guestbook submission through Burp. Together, they support the stored-input demonstration in the local lab.

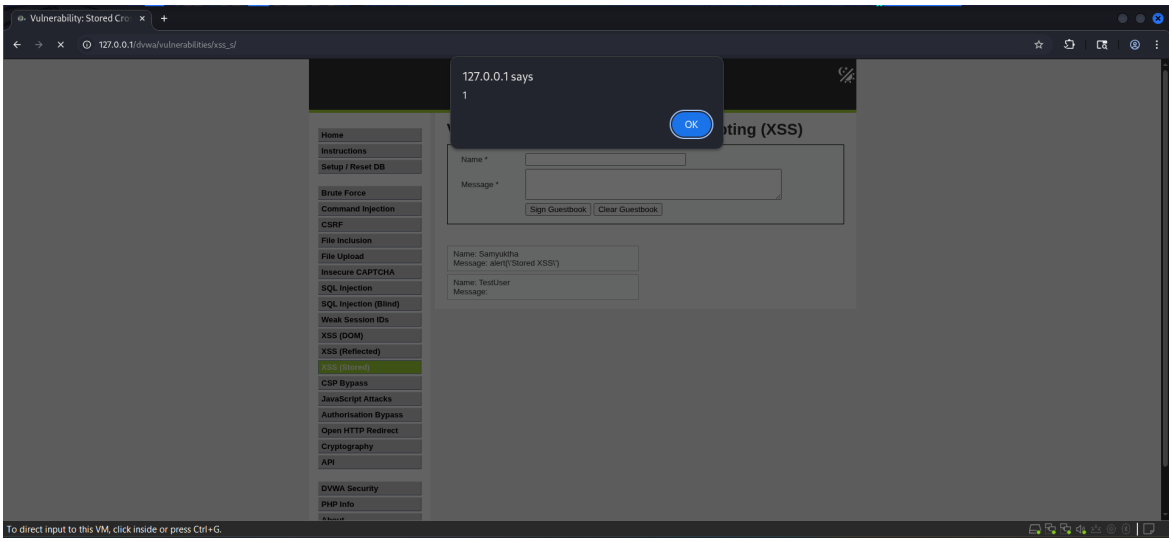


Figure 8. Stored XSS browser confirmation.

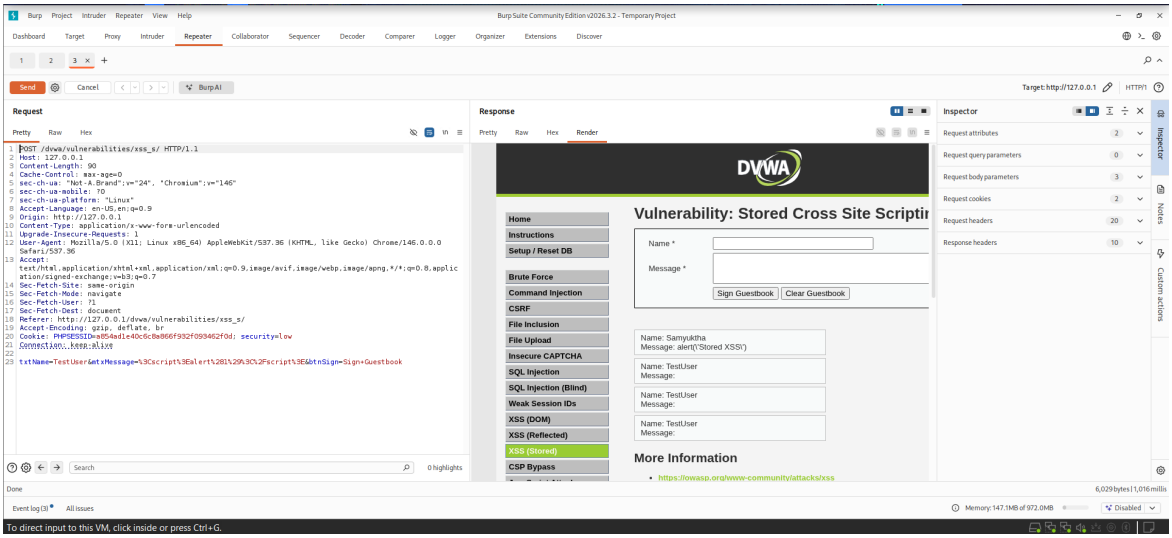


Figure 9. Burp evidence for stored message submission.

6.4 Intruder Position Configuration

A local DVWA login POST request was sent to Intruder and the username value was selected as the payload position. The screenshot documents configuration only. No password-guessing attack was run, and no credential-guessing or authentication-bypass result is claimed.

6.4–6.5 Additional Burp Tests

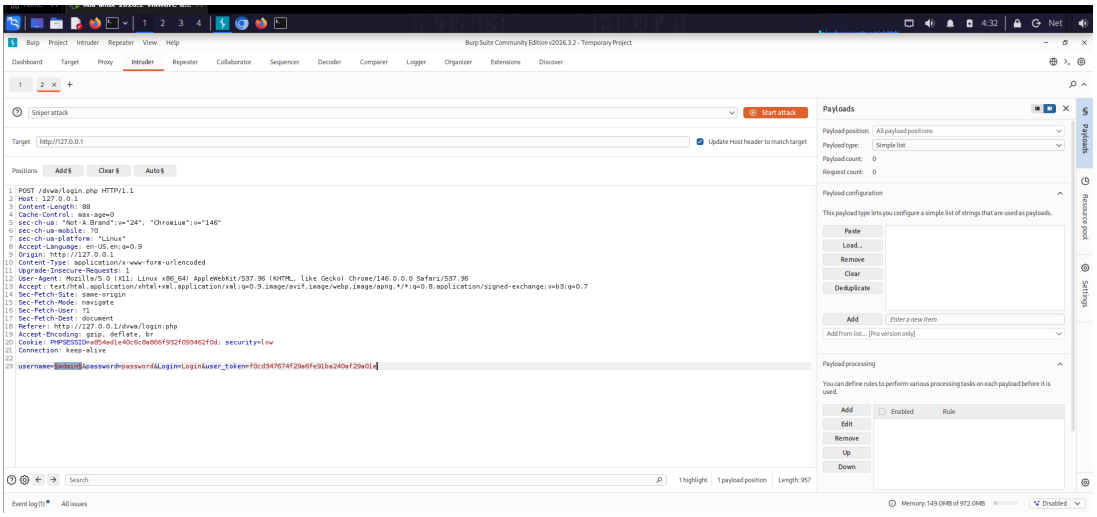


Figure 10. Username payload position marked in Intruder; no attack launched.

6.5 Parameter Change / Repeater Comparison

The ID parameter was changed from 1 to 2 in a request sent through Burp Repeater, and the application returned a response. This demonstrates request modification and response comparison. It does not independently prove broken access control: that would require evidence that a user accessed a resource they were not authorized to view, under a defined account and permission model.

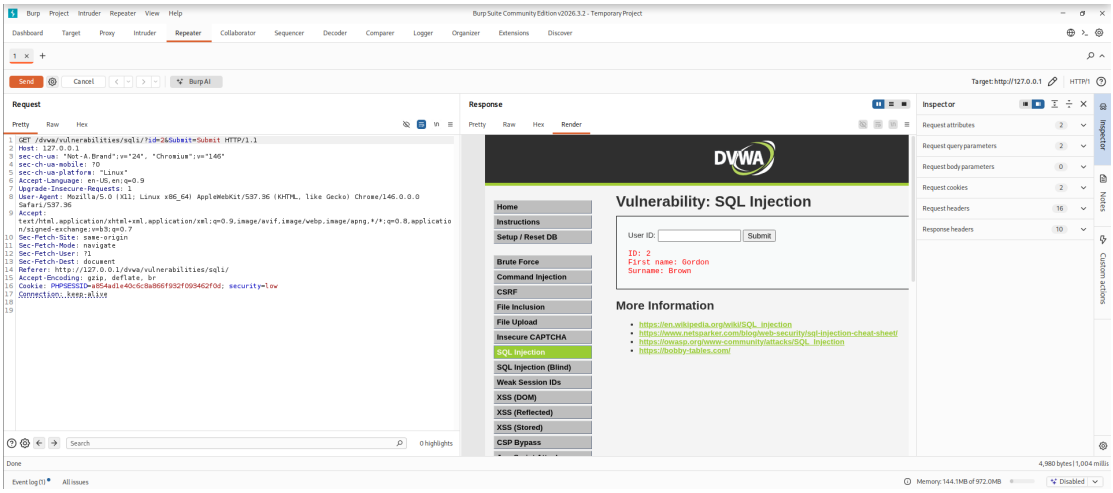


Figure 11. Repeater comparison after changing ID from 1 to 2.

7. Endpoint Discovery

Manual browsing and the Burp site map were used to record endpoints. Automated directory discovery with dirsearch or Gobuster is not documented in the evidence, so the inventory below is described as manually observed rather than exhaustive.

7.1 Endpoint Map and Findings Register

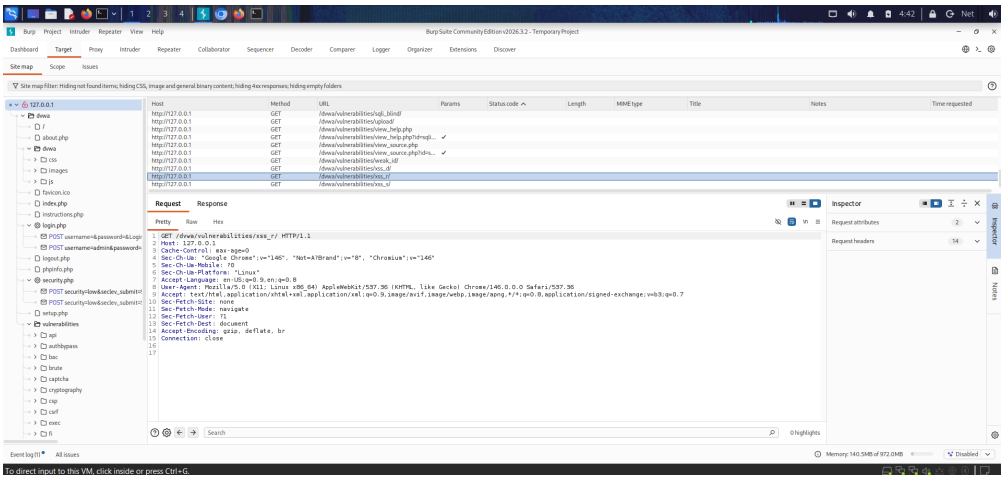


Figure 12. Local DVWA endpoint map captured in Burp.

Finding / observation	Evidence and interpretation	Status
SQL Injection	Controlled input returned multiple user records in DVWA Low.	Demonstrated in local lab
Reflected XSS	Alert executed after reflected input; related Burp evidence saved.	Demonstrated in local lab
Stored XSS	Stored guestbook entry triggered alert when rendered.	Demonstrated in local lab
Intruder setup	Username position selected; no payload attack launched.	Configuration only
Parameter change	ID changed from 1 to 2; response received.	Not independently confirmed as a vulnerability
Endpoint discovery	Routes recorded through manual browsing and site map.	Inventory only

8. Remediation Recommendations

- SQL injection: prepared statements, least-privilege database access, safe error handling and regression tests.
- Reflected/stored XSS: context-aware output encoding, safe rendering APIs and allowlist sanitization where rich HTML is required.
- Defense in depth: deploy a restrictive Content Security Policy after checking application compatibility.
- Validation: rerun the same controlled test cases after changes and retain before/after request-response evidence.

9. Evidence Index

Evidence	File
Login POST	01-Setup-Interception/01-Burp-Login-POST.png
HTTPS observation	01-Setup-Interception/02-Burp-HTTPS-Interception.png
DVWA site map	01-Setup-Interception/03-DVWA-Site-Map.png
SQLi baseline / result	03-Vulnerability-Testing/04-SQLi-Baseline-Request.png; 05-SQLi-Confirmed.png
Reflected XSS	03-Vulnerability-Testing/06-Reflected-XSS-Confirmed.png; 07-Reflected-XSS-Burp-Evidence.png
Stored XSS	03-Vulnerability-Testing/08-Stored-XSS-Confirmed.png; 09-Stored-XSS-Burp-Request-Response.png
Intruder / parameter comparison	03-Vulnerability-Testing/10-Intruder-Positions.png; 11-Parameter-Tampering.png
Endpoint map	04-Endpoint-Discovery/12-DVWA-Endpoint-Map.png
Text notes	02-Recon-Mapping/Attack-Surface.txt; 04-Endpoint-Discovery/Endpoint-Notes.txt

10. Learning Outcomes

- Used Burp Proxy and HTTP history to observe browser-generated requests.
- Learned to use the Target site map as an initial application-route inventory.
- Compared a baseline request with a controlled SQLi test input and documented the resulting behavior.
- Distinguished reflected from stored XSS through how the test input was returned or saved and rendered.
- Used Repeater to modify a request and understood why a response alone does not establish unauthorized access.
- Recorded Intruder position setup accurately without claiming an attack that was not run.

11. Conclusion

The Week 7 practical exercise documented three demonstrated behaviors in local DVWA: SQL injection, reflected XSS and stored XSS. Burp was also used for interception, endpoint mapping, Intruder position configuration and request comparison. The findings and limitations are separated so the report remains evidence-based. The next useful step is to validate recommended fixes in the lab and retain comparable before/after evidence.

12. References

- PortSwigger Web Security Academy — learning material for Burp Suite and web application testing: https://portswigger.net/web-security
- OWASP Cheat Sheet Series — SQL Injection Prevention: https://cheatsheetseries.owasp.org/
- OWASP Cheat Sheet Series — Cross Site Scripting Prevention: https://cheatsheetseries.owasp.org/
- Damn Vulnerable Web Application (DVWA) — local intentionally vulnerable training application.

Submission hygiene: before sharing screenshots publicly, redact session cookies, PHPSESSID values, tokens, usernames/passwords and other sensitive values if visible.